

TN SKILLS PROJECT DOCUMENTATION

FITBUDDY

AI-Powered Personalized Fitness Plan Generator using Google Gemini Models

A Project Report Submitted in Partial Fulfilment of the
Requirements of the TN Skills Project Programme

Submitted by

Tarunya V (222502806) — Team Leader

Stellamary (222502805)

Abirami R (222502776)

Sripriya S (222502804)

Varsha S (222502809)

Abstract

Maintaining a consistent and effective fitness routine is a challenge for most individuals due to the lack of personalised guidance, the cost of professional trainers, and the generic nature of fitness content available online. FitBuddy is an AI-powered fitness web application designed to bridge this gap by generating a personalised seven-day workout plan, using Google's Gemini generative AI models. Unlike static fitness applications that rely on pre-defined templates, FitBuddy dynamically constructs a workout plan based on a user's name, age, weight, fitness goal and preferred workout intensity.

The system follows a simple, layered architecture consisting of a server-rendered HTML/CSS frontend built with Jinja2 templates, a Python FastAPI backend that exposes a small set of web routes, a SQLite database accessed through SQLAlchemy for persistent storage of user profiles and generated plans, and an integration layer that communicates with the Gemini API for natural-language plan generation. The backend constructs a structured prompt from the user's profile, sends it to the Gemini model, and renders the model's response as a day-by-day workout plan on the result page, alongside a separately generated nutrition and recovery tip relevant to the user's goal.

The application supports collection of user information, AI-generated plan creation, a feedback mechanism through which a user can request changes to their plan and receive an updated version, persistent storage of both the original and updated plans, and a password-protected administrator dashboard for viewing all registered users and their stored plans. The use of Gemini models allows FitBuddy to produce plans that read naturally and adapt to the user's stated goal and intensity, without requiring the application's own codebase to be retrained.

This documentation presents, in detail, the problem definition, literature survey, system analysis, requirement specification, system architecture, frontend design, backend design, database schema, Gemini AI integration strategy, module-wise implementation, testing methodology and results of the FitBuddy project. It concludes with the advantages, limitations, and scope for future enhancement of the system, including user accounts, progress tracking and a conversational follow-up assistant.

Keywords

Artificial Intelligence, Generative AI, Google Gemini API, Fitness Recommendation System, Personalised Workout Plan, FastAPI, SQLAlchemy, SQLite, Jinja2, Python Web Application.

Table of Contents

Abstract	2
Chapter 1 – Introduction	4
Chapter 2 – Literature Survey	6
Chapter 3 – System Analysis	8
Chapter 4 – System Requirements Specification	10
Chapter 5 – System Design	12
Chapter 6 – Technology Stack	15
Chapter 7 – Frontend Design and Implementation	17
Chapter 8 – Backend Design and Implementation	19
Chapter 9 – Gemini AI Integration	22
Chapter 10 – Module-wise Implementation	24
Chapter 11 – Testing	26
Chapter 12 – Results and Discussion	28
Chapter 13 – Advantages and Limitations	29
Chapter 14 – Future Enhancements	30
Chapter 15 – Conclusion	31
References	32
Appendix	33

Chapter 1 – Introduction

1.1 Overview

Physical fitness and healthy living have become central concerns in modern lifestyles, yet a large section of the population struggles to adopt and sustain an effective workout routine. The main barriers are the absence of personalised guidance, the cost of hiring a certified personal trainer, and the overwhelming amount of generic, often contradictory, fitness content available online. FitBuddy is conceived as a solution to this problem — a lightweight, AI-driven fitness companion web application that generates an individually tailored seven-day workout plan in seconds, using Google's Gemini generative AI models as its reasoning engine.

The project combines a simple, fast Python web backend built on FastAPI with applied generative AI (prompt engineering over the Gemini API) to deliver a product that feels like a personal trainer available on demand. FitBuddy does not retrieve a plan from a fixed library of templates; instead, it builds a natural-language prompt from the user's own profile — their name, age, weight, fitness goal and preferred workout intensity — and lets the Gemini model reason over that context to produce a plan that is specific to the individual. Users can then react to the plan they receive: FitBuddy accepts free-text feedback and uses it, together with the original plan and the user's profile, to generate a revised plan through a second call to Gemini.

1.2 Problem Statement

Existing fitness applications typically fall into one of two categories. The first consists of simple workout-tracking apps that log exercises and repetitions but provide no intelligent recommendation of what a user should actually do. The second consists of subscription coaching platforms that do offer personalisation, but at a high recurring cost, and that personalisation is performed by a limited pool of human coaches who cannot scale to serve a very large user base with genuinely individual attention. There is a clear gap for a system that can generate a personalised, goal-oriented fitness plan automatically, at low cost, that can also be refined through a simple feedback loop — without requiring a subscription, a mobile app install, or a human coach in the loop.

1.3 Objectives of the Project

- To design and develop a lightweight full-stack web application that collects a user's basic fitness information through a simple web form.
- To integrate Google's Gemini generative AI models to dynamically generate a structured, easy-to-follow seven-day workout plan based on the user's profile, fitness goal and workout intensity.
- To generate a complementary nutrition and recovery tip tailored to the user's fitness goal using a second, lighter Gemini model call.
- To allow users to submit feedback on the plan they receive and to regenerate an updated plan that incorporates that feedback.
- To persist user information and both the original and updated workout plans in a relational database for later retrieval.
- To provide a secure administrator login and a dashboard through which an administrator

can view all registered users and their stored plans.

- To build the system using a simple, well-documented and industry-standard technology stack (FastAPI, SQLAlchemy, SQLite and Jinja2) that is easy to run locally and easy to extend.
- To document the complete software development lifecycle of the project for academic evaluation under the TN Skills Project programme.

1.4 Scope of the Project

The scope of FitBuddy, in its current version, covers the collection of a user's personal fitness information, AI-generated seven-day workout plan creation using the Gemini API, an AI-generated nutrition and recovery tip, a feedback form that triggers regeneration of the plan, storage of users and their plans in a SQLite database, and a password-protected administrator dashboard for viewing all registered users and plans. The scope explicitly excludes user account login/registration (users identify themselves with a simple user ID rather than a password-based account), real-time video-based exercise form correction, integration with wearable hardware, and medical diagnosis – FitBuddy is a fitness and wellness guidance tool and is not a substitute for professional medical advice. Several of these excluded items are discussed further under Future Enhancements in Chapter 14.

1.5 Organization of the Report

This report is organised into fifteen chapters. Chapter 2 surveys existing literature and related fitness applications. Chapter 3 presents the system analysis, comparing the existing and proposed systems. Chapter 4 specifies the hardware, software, functional and non-functional requirements. Chapter 5 details the system design, including the architecture, data flow and database design. Chapter 6 describes the technology stack in depth. Chapters 7 and 8 provide a detailed account of the frontend and backend implementation respectively. Chapter 9 focuses specifically on the Gemini AI integration and prompt design. Chapter 10 describes the implementation on a module-by-module basis. Chapter 11 covers the testing methodology and test cases. Chapter 12 presents the results. Chapter 13 discusses advantages and limitations, Chapter 14 outlines future enhancements, and Chapter 15 concludes the report, followed by references and an appendix.

Chapter 2 — Literature Survey

2.1 Introduction to the Domain

The intersection of health informatics and artificial intelligence has produced a wide variety of digital fitness solutions over the last decade, ranging from simple pedometer-based step counters to sophisticated coaching platforms that adjust training load using machine learning. Before designing FitBuddy, a survey of existing commercial products and academic literature was carried out to understand the strengths and shortcomings of current approaches.

2.2 Survey of Existing Fitness Applications

2.2.1 Template-Based Workout Apps

A large majority of fitness applications available on app stores rely on a fixed library of pre-authored workout templates that are filtered by a small number of parameters such as goal (weight loss, muscle gain) and experience level. While simple and reliable, the plans they generate are not truly personalised — two users with very different profiles may receive an identical plan simply because they selected the same goal.

2.2.2 Human-Coached Subscription Platforms

Several platforms connect users with certified human trainers who design and adjust plans manually. These platforms offer genuine personalisation but do not scale economically, since the cost of a coach's time is passed on to the user as a recurring subscription fee.

2.2.3 Rule-Based Recommendation Engines

Some tools use rule-based expert systems (if-then logic built on goal categories and intensity levels) to recommend plans. These are more personalised than pure templates but are limited by the number of rules developers can practically author and maintain, and they struggle with unusual combinations of inputs.

2.2.4 Generative-AI-Based Fitness Assistants

A recent generation of fitness tools uses large language models to generate plans conversationally. These tools demonstrate that a well-designed prompt built from a user's profile can produce plans that are coherent and well-reasoned, while requiring far less manual engineering effort than a rule-based system, because the underlying model has already been trained on a vast corpus of general fitness knowledge. FitBuddy adopts this generative-AI approach, using Google's Gemini models.

2.3 Comparative Summary

Approach	Personalisation	Scalability	Development Effort	Feedback-Driven Refinement
Template-based apps	Low	High	Low	No
Human-coached platforms	High	Low	High (ongoing)	Yes, manually
Rule-based engines	Medium	High	High (rule authoring)	Limited

The survey shows a gap for a system that combines the low cost and scalability of a template-based app with the personalisation of a human coach, and that additionally allows the user to refine the output through plain-language feedback rather than re-filling an entire form. FitBuddy addresses this gap with a minimal, form-driven interface backed by Gemini, and a dedicated feedback-to-regeneration pipeline.

Chapter 3 — System Analysis

3.1 Existing System

Before FitBuddy, a user wanting a workout plan would typically either follow a generic plan found online or in a fitness app, hire a personal trainer, or manually research exercises appropriate to their goal. Generic plans do not account for the individual's age, weight, stated goal or preferred intensity; hiring a trainer is costly and not always accessible; and self-research is time-consuming and requires a degree of fitness knowledge that many users do not have.

3.2 Proposed System

FitBuddy proposes a self-service web application in which a user supplies a small set of personal details — name, user ID, age, weight, fitness goal and workout intensity — through a single web form. The FastAPI backend forwards this information, inside a carefully constructed prompt, to the Gemini API, which returns a structured seven-day workout plan. A second, lighter Gemini call produces a nutrition and recovery tip related to the user's goal. Both the user's information and the generated plan are stored in a SQLite database. If the user is not satisfied with the plan, they can submit feedback describing what they would like changed; FitBuddy then constructs a second prompt that includes the original plan, the feedback and the user's profile, and asks Gemini to produce an updated plan, which is also stored. An administrator, authenticated through credentials configured as environment variables, can log in to a dashboard that lists every registered user together with their stored original plan, updated plan, nutrition tip and feedback.

3.3 Feasibility Study

3.3.1 Technical Feasibility

All the technologies used in FitBuddy — Python, FastAPI, SQLAlchemy, SQLite, Jinja2 and the Gemini API via the google-genai client library — are mature, well-documented, and freely available. FastAPI's asynchronous request handling and automatic request validation make it straightforward to build the small number of routes FitBuddy requires, and SQLite removes the need to install or administer a separate database server, which is well suited to an academic project of this scope.

3.3.2 Economic Feasibility

The project uses only open-source or free-tier components. The only recurring cost is usage of the Gemini API, which offers a free tier sufficient for development and demonstration purposes. No licensing fees are required for any part of the stack.

3.3.3 Operational Feasibility

The application is operated through a single command (`uvicorn app.main:app --reload`) and requires no installation beyond a Python virtual environment and the packages listed in `requirements.txt`. The browser-based interface requires no client-side installation, which keeps the system simple to run and demonstrate.

3.4 Requirement Gathering Approach

Requirements were gathered by identifying the smallest set of inputs that would allow Gemini to produce a genuinely personalised plan (name, age, weight, goal and intensity), by reviewing common features of existing fitness apps described in the literature survey, and by considering the practical needs of an academic evaluator, namely a way to inspect stored user data without direct database access — which motivated the administrator dashboard.

Chapter 4 — System Requirements Specification

4.1 Hardware Requirements

Component	Minimum Specification
Processor	Dual-core 16 GHz or higher
RAM	4 GB (8 GB recommended)
Storage	500 MB free disk space (SQLite database grows with usage)
Network	Active internet connection (required for Gemini API calls)
Display	Any device with a modern web browser

Table 4.1: Minimum hardware requirements

4.2 Software Requirements

Category	Software / Package
Operating System	Windows, macOS or Linux
Language Runtime	Python 3.x
Web Framework	FastAPI
ASGI Server	Uvicorn
ORM	SQLAlchemy
Database	SQLite
Templating	Jinja2
AI SDK	google-generativeai (Google Gemini API client)
Other Python Packages	python-dotenv, python-multipart
Development Tools	Visual Studio Code, Git, GitHub, Python virtual environment

Table 4.2: Software requirements

4.3 Functional Requirements

- The system shall allow a user to enter personal information (name, user ID, age, weight).
- The system shall allow a user to select a fitness goal and a workout intensity from a predefined set of options.
- The system shall generate a personalised seven-day workout plan by sending the user's profile to the Gemini API.
- The system shall generate a nutrition/recovery tip related to the user's fitness goal using the Gemini API.
- The system shall display the generated workout plan and nutrition tip to the user.
- The system shall allow the user to submit free-text feedback about the generated plan.
- The system shall generate an updated workout plan using the original plan, the feedback and the user's profile.
- The system shall store the user's information, the original plan, the updated plan, the nutrition tip and the feedback in a database.

- The system shall provide a secure login for the administrator, using credentials configured through environment variables.
- The system shall allow the administrator to view all registered users and their stored plans through a dashboard.
- The system shall allow the administrator to log out of the dashboard.

4.4 Non-Functional Requirements

- Usability — the interface shall be simple enough for a first-time user to generate a plan without instructions, using a single form.
- Performance — a workout plan shall typically be returned within a few seconds of form submission, subject to Gemini API response time.
- Security — administrator credentials and the Gemini API key shall never be hard-coded and shall instead be read from environment variables (.env).
- Reliability — user and plan data shall persist across application restarts through the SQLite database file.
- Portability — the application shall run on any operating system that supports Python 3.x and shall require no dependency beyond the packages listed in requirements.txt.
- Maintainability — the codebase shall separate concerns across dedicated modules (routing, database access, administrator authentication, and the three Gemini generator modules) to keep each file small and focused.

Chapter 5 — System Design

5.1 System Architecture

FitBuddy follows a simple layered architecture. A user interacts with a server-rendered HTML/CSS interface built with Jinja2 templates. Every user action — submitting the personal-information form, submitting feedback, or an administrator logging in — is handled by a route in the FastAPI backend. The backend communicates with two external-facing components: a SQLite database (accessed through SQLAlchemy) for persistent storage, and the Gemini API for AI-generated content. The diagram below summarises this architecture.

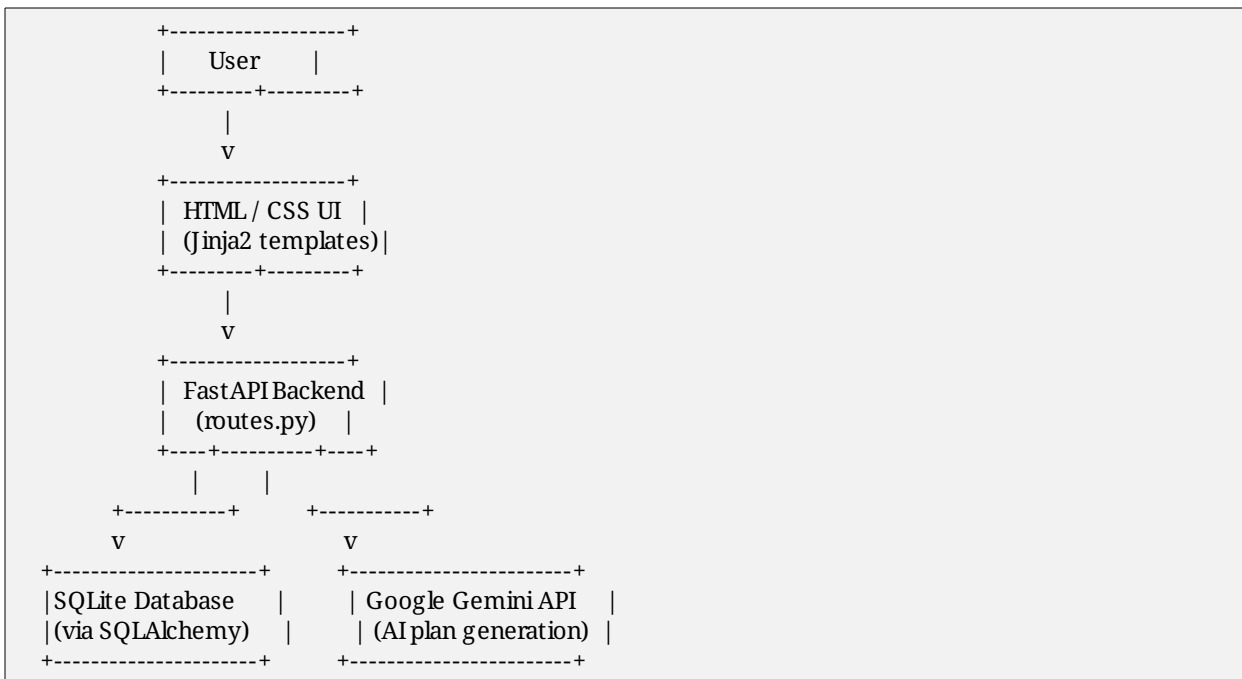


Figure 5.1 FitBuddy system architecture

5.2 Data Flow Diagram

The primary data flow begins when the user submits the personal-information form. `routes.py` validates and forwards this data to `gemini_generator.py`, which builds a prompt and calls the Gemini API; the returned plan is passed to `database.py` for storage and then rendered back to the user together with a nutrition tip produced by `gemini_flash_generator.py`. When the user submits feedback, `routes.py` retrieves the stored original plan and user information from the database and passes them, together with the feedback text, to `updated_plan.py`, which calls Gemini again and stores the resulting updated plan. The administrator flow is simpler: `admin_auth.py` validates the submitted credentials against the values configured in the environment, and on success `routes.py` queries `database.py` for every user and plan and renders them in the `all_users.html` dashboard.

5.3 Entity-Relationship (Database) Design

Table	Field	Description
users	id	Primary key (auto-increment)

users	user_id	User-supplied identifier used to link a user to their plans
users	username	Name entered by the user
users	age	Age entered by the user
users	weight	Weight entered by the user
users	goal	Selected fitness goal
users	intensity	Selected workout intensity
plans	id	Primary key (auto-increment)
plans	user_id	Foreign key referencing users.user_id
plans	original_plan	The first AI-generated seven-day workout plan
plans	updated_plan	The plan regenerated after user feedback
plans	nutrition_tip	AI-generated nutrition/recovery tip
plans	feedback	Free-text feedback supplied by the user

Table 5.1 users and plans table design

The users and plans tables are related one-to-one through `user_id`: each registered user has at most one stored plan record, which is updated in place with a new `original_plan`, `updated_plan`, `nutrition_tip` or `feedback` value as the user progresses through the workflow.

5.4 Use Case Diagram (Description)

FitBuddy has two actors: the Registered User and the Administrator. Their interactions with the system are summarised as a list of use cases below, since the workflow is linear and is described more clearly this way than as a graphical diagram.

- Enter Personal Information — the user supplies name, user ID, age and weight.
- Select Fitness Goal and Workout Intensity — the user chooses from predefined options.
- Generate Workout Plan — the system calls Gemini and displays a seven-day plan and a nutrition tip.
- Submit Feedback — the user describes changes they would like to see in the plan.
- View Updated Plan — the system calls Gemini again and displays the revised plan (extends Generate Workout Plan).
- Administrator Login — the administrator authenticates using environment-configured credentials.
- View All Users and Plans — the administrator views every registered user's stored data on a dashboard.
- Administrator Logout — the administrator ends their session.

5.5 Module Design Overview

The backend is organised into a small number of cooperating modules, each responsible for one concern:

- `main.py` — application entry point, static files, templates and route registration.
- `routes.py` — all web routes: home page, plan generation, feedback, plan update,

administrator login/logout and the users dashboard.

- `database.py` — SQLAlchemy models, session management and all create/read/update operations on the users and plans tables.
- `admin_auth.py` — validation of administrator credentials against environment variables.
- `gemini_generator.py` — builds the prompt for, and calls, Gemini to generate the original seven-day workout plan.
- `gemini_flash_generator.py` — builds the prompt for, and calls, Gemini to generate the nutrition/recovery tip.
- `updated_plan.py` — builds the prompt for, and calls, Gemini to generate an updated plan from feedback.
- `schemas.py` — request/response data structures used across the routes.

5.6 UI Flow

At a high level, the frontend screens flow in the following order for a first-time user:

1. Home Page (`index.html`) — the user enters personal information and selects a fitness goal and workout intensity.
2. Plan Generation — the form is submitted to the backend, which calls Gemini and stores the result.
3. Result Page (`result.html`) — displays the seven-day workout plan and the nutrition/recovery tip.
4. Feedback Page (`feedback.html`) — allows the user to describe changes they would like made to the plan.
5. Updated Result — the regenerated plan is displayed to the user.
6. Administrator Login Page (`admin_login.html`) — a separate entry point for the administrator.
7. Administrator Dashboard (`all_users.html`) — lists every registered user and their stored plan data.

Chapter 6 — Technology Stack

6.1 Overview

FitBuddy deliberately uses a small, well-understood technology stack rather than a large framework ecosystem, keeping the codebase easy to read, run and extend within the scope of an academic project. The stack is organised into frontend technologies, backend technologies, the Gemini AI engine, the database, and supporting development tools.

6.2 Frontend Technologies

- HTML5 — structure of every page served by the application.
- CSS3 — styling for a responsive, single-column layout that works on both desktop and mobile browsers.
- Jinja2 Templates — server-side templating used by FastAPI to render `index.html`, `result.html`, `feedback.html`, `admin_login.html` and `all_users.html` with dynamic data.

6.3 Backend Technologies

- Python 3.x — the implementation language for the entire backend.
- FastAPI — the web framework used to define routes, validate request data and render templates.
- Uvicorn — the ASGI server used to run the FastAPI application.
- SQLAlchemy — the ORM used to define the users and plans models and to query the SQLite database.
- python-dotenv — loads environment variables (administrator credentials, Gemini API key) from a `.env` file.
- python-multipart — enables FastAPI to parse HTML form submissions.

6.4 AI Engine — Google Gemini

FitBuddy uses Google's Gemini generative AI models, accessed through the official `google-generativeai` Python client library, for all AI-generated content. Two Gemini calls are made per plan-generation cycle: one to `gemini_generator.py` to produce the seven-day workout plan, and one to `gemini_flash_generator.py` to produce a lighter-weight nutrition/recovery tip. A third call, in `updated_plan.py`, regenerates the plan when the user submits feedback. Using a hosted generative model, rather than a locally trained one, means FitBuddy benefits from continual model improvements without any retraining of its own codebase.

6.5 Database

SQLite was chosen as the database engine because it requires no separate server process, stores the entire database as a single file (`fitbuddy.db`) that travels with the project, and is fully supported by SQLAlchemy. This makes FitBuddy simple to set up for grading and demonstration, while still exercising a real relational schema with two related tables.

6.6 Development and Deployment Tools

- Visual Studio Code — primary code editor used throughout development.
- Git and GitHub — version control and source hosting for the project repository.
- Python Virtual Environment (venv) — isolates project dependencies from the system Python installation.

Chapter 7 — Frontend Design and Implementation

7.1 Frontend Project Structure

```

templates/
  index.html      # Home page: personal info + goal + intensity form
  result.html     # Displays the generated 7-day plan and nutrition tip
  feedback.html   # Form for the user to submit feedback on the plan
  admin_login.html # Administrator login form
  all_users.html  # Administrator dashboard listing users and plans

static/
  images/
    gym_bg_1.jpg  # Background image used across the interface

```

Figure 7.1 Frontend template and static-asset structure

7.2 Key Pages and Screens

Template	Purpose
index.html	Collects username, user ID, age, weight, fitness goal and workout intensity
result.html	Renders the AI-generated seven-day workout plan and the nutrition/recovery tip
feedback.html	Collects free-text feedback about the displayed plan
admin_login.html	Collects administrator username and password
all_users.html	Lists every registered user together with their stored plan, updated plan, nutrition tip and feedback

Table 7.1 Purpose of each Jinja2 template

7.3 Template Rendering and Static Assets

FastAPI's Jinja2Templates integration is configured once in `main.py` and reused by every route in `routes.py` that needs to render a page. Static assets (currently a single background image) are served through FastAPI's StaticFiles mount, keeping presentation assets separate from the Python source tree. Because rendering happens on the server, no client-side JavaScript framework is required and every page works correctly even with JavaScript disabled in the browser.

7.4 Form Handling and Data Flow

Every page that collects input from the user — the home page, the feedback page and the administrator login page — uses a standard HTML `<form>` that performs an HTTP POST back to a FastAPI route. `python-multipart` allows FastAPI to parse these submissions directly into typed function parameters, which are validated before being passed on to the database and Gemini generator modules. This keeps the frontend free of client-side validation logic, with FastAPI's own request validation acting as the single source of truth.

7.5 UI / UX Design Principles

- Single-form simplicity — all personal information, goal and intensity are collected on one page to minimise steps.
- Immediate feedback loop — the result page leads directly into a feedback form, so refining a plan takes one extra step rather than starting over.
- Responsive layout — CSS3 is used to ensure the interface remains usable on both desktop and mobile screen widths.
- Clear separation of user and administrator views — the administrator dashboard is reachable only through its own login page, keeping user-facing and administrative screens distinct.

7.6 Sample Frontend Code — Goal and Intensity Selection

```
<!-- templates/index.html (excerpt) -->
<form action="/generate-plan" method="post">
  <input type="text" name="username" placeholder="Your name" required>
  <input type="text" name="user_id" placeholder="User ID" required>
  <input type="number" name="age" placeholder="Age" required>
  <input type="number" name="weight" placeholder="Weight (kg)" required>

  <select name="goal" required>
    <option value="weight_loss">Weight Loss</option>
    <option value="muscle_gain">Muscle Gain</option>
    <option value="endurance">Endurance</option>
  </select>

  <select name="intensity" required>
    <option value="low">Low</option>
    <option value="medium">Medium</option>
    <option value="high">High</option>
  </select>

  <button type="submit">Generate My Plan</button>
</form>
```

Figure 7.2: Simplified excerpt of the home-page form

Chapter 8 — Backend Design and Implementation

8.1 Backend Project Structure

```

app/
__init__.py
main.py          # App creation, static files, templates, route registration
routes.py        # All web routes
database.py      # SQLAlchemy models and DB operations
admin_auth.py    # Administrator credential validation
gemini_generator.py  # Original 7-day plan generation
gemini_flash_generator.py  # Nutrition / recovery tip generation
updated_plan.py  # Feedback-driven plan regeneration
schemas.py       # Request / response data structures

```

Figure 8.1 Backend application structure

8.2 Application Routes

Route	Method	Description
/	GET	Displays the home page with the personal-information and goal/intensity form
/generate-plan	POST	Accepts form data, calls Gemini, stores and displays the plan and nutrition tip
/feedback	GET	Displays the feedback form for the most recently generated plan
/update-plan	POST	Accepts feedback, calls Gemini again and displays the updated plan
/admin/login	GET / POST	Displays the administrator login form and validates submitted credentials
/admin/users	GET	Displays the dashboard listing all registered users and their plans
/admin/logout	GET	Ends the administrator session
/health	GET	Health-check endpoint used to confirm the API is running

Table 8.1 FitBuddy application routes (routes.py)

8.3 Administrator Authentication

Administrator credentials are never hard-coded into the application. Instead, an administrator username and password are configured as environment variables and loaded through `python-dotenv` at start-up. When the administrator submits the login form, `admin_auth.py` compares the submitted values against these environment-configured credentials. On success, the administrator is granted access to the `/admin/users` dashboard for the remainder of the session; on failure, the login page is re-rendered with an error message. This approach keeps sensitive credentials out of the source code and out of version control, since the `.env` file is excluded through `.gitignore`.

```

# app/admin_auth.py (excerpt)
import os

```

```
def verify_admin(username: str, password: str) -> bool:
    return (
        username == os.getenv("ADMIN_USERNAME")
        and password == os.getenv("ADMIN_PASSWORD")
    )
```

Figure 8.2: Administrator credential verification

8.4 Database Schema (SQLAlchemy Models)

```
# app/database.py (excerpt)
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String, unique=True, index=True)
    username = Column(String)
    age = Column(Integer)
    weight = Column(Float)
    goal = Column(String)
    intensity = Column(String)

class Plan(Base):
    __tablename__ = "plans"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String, ForeignKey("users.user_id"))
    original_plan = Column(Text)
    updated_plan = Column(Text)
    nutrition_tip = Column(Text)
    feedback = Column(Text)
```

Figure 8.3: SQLAlchemy models for the users and plans tables

8.5 Workout Plan Generation Service

`gemini_generator.py` is responsible for turning a user's profile into a Gemini prompt and returning a structured seven-day plan. It accepts the username, age, weight, fitness goal and workout intensity, formats these into a natural-language instruction that asks Gemini for a day-by-day breakdown of exercises, and returns the model's response as plain text ready to be stored and rendered. `gemini_flash_generator.py` follows the same pattern but with a much shorter prompt that asks only for a short nutrition or recovery tip relevant to the user's goal, keeping this second call fast and inexpensive. `updated_plan.py` extends the same approach by including the previously generated plan and the user's feedback text in the prompt, asking Gemini to revise the plan rather than create a new one from scratch.

8.6 Error Handling and Logging

Routes that call external services (the Gemini API) or perform database operations are wrapped in exception handling so that a failed AI call or a database error results in a clear, user-facing message rather than an unhandled server error. FastAPI's built-in request validation additionally rejects malformed form submissions (for example, a non-numeric age) before they reach the business logic, returning a descriptive validation error automatically.

8.7 Environment Configuration

--	--

Variable	Purpose
ADMIN_USERNAME	Administrator login username, checked by admin_auth.py
ADMIN_PASSWORD	Administrator login password, checked by admin_auth.py
DATABASE_URL	SQLite connection string used by SQLAlchemy (defaults to the local fitbuddy.db file)

Table 8.2: Environment variables defined in .env / .env.example

Chapter 9 — Gemini AI Integration

9.1 Overview of Gemini Models

Google Gemini is a family of multimodal generative AI models accessible through the google-genai Python client library. FitBuddy uses Gemini purely as a text-generation engine: structured natural-language prompts are sent to the API and the model's text response is parsed and displayed, with no image or audio input involved.

9.2 Why Gemini was chosen for FitBuddy

Gemini was selected because it is accessible through an official, well-maintained Python SDK, offers a usable free tier suitable for an academic project, and produces coherent, well-formatted long-form text that is well suited to a day-by-day workout plan. Because the reasoning is performed by a hosted model rather than a locally trained one, FitBuddy's own codebase never needs to be retrained as the underlying model improves.

9.3 Gemini Client Setup

```
# app/gemini_generator.py (excerpt)
from google import genai
import os

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
```

Figure 9.1 Gemini client initialisation

9.4 Prompt Engineering — Workout Plan Generation

The prompt sent to Gemini for the original plan embeds the user's name, age, weight, fitness goal and workout intensity, and explicitly asks for a day-by-day (Day 1 to Day 7) breakdown so that the response can be displayed directly without additional post-processing.

```
prompt = f"""
You are a certified fitness coach. Create a personalised 7-day
workout plan for the following person:

Name: {username}
Age: {age}
Weight: {weight} kg
Fitness Goal: {goal}
Workout Intensity: {intensity}

Present the plan as Day 1 through Day 7, with specific exercises,
sets and repetitions appropriate to the stated goal and intensity.
"""
```

Figure 9.2: Prompt template used by gemini_generator.py

9.5 Invoking Gemini and Parsing the Response

```
response = client.models.generate_content(
    model="gemini-1.5-flash",
    contents=prompt,
)
```

```
plan_text = response.text
```

Figure 9.3: Calling the Gemini API and extracting the plan text

9.6 Prompt Engineering — Nutrition/Recovery Tip Generation

gemini_flash_generator.py sends a much shorter prompt — essentially, 'Give a concise nutrition or recovery tip for someone whose fitness goal is {goal}' — so that this second call returns quickly and keeps the overall page-generation time low.

9.7 Prompt Engineering — Updated Plan Generation from Feedback

```
prompt = f"""
Here is a 7-day workout plan previously generated for {username}
(goal: {goal}, intensity: {intensity}):

{original_plan}

The user has given the following feedback:
"{feedback}"

Revise the plan to address this feedback while keeping it aligned
with the user's original goal and intensity. Present the revised
plan as Day 1through Day 7.
"""
```

Figure 9.4: Prompt template used by updated_plan.py

9.8 Handling AI Response Variability

Because Gemini's output is generated text rather than a fixed schema, FitBuddy's prompts are written to consistently request a Day 1 to Day 7 structure so that the response can be stored and displayed as-is, without brittle parsing logic. If the API call fails or times out, the affected route catches the exception and returns a clear error message to the user instead of a raw server error.

Chapter 10 — Module-wise Implementation

10.1 User Information Module

Implemented across `index.html` and the relevant handler in `routes.py`, this module collects the user's name, user ID, age and weight and passes them, together with the selected goal and intensity, into the plan-generation flow. `database.py` persists this information as a row in the `users` table, keyed by `user_id`.

10.2 Fitness Goal and Intensity Selection Module

A pair of dropdown selectors on the home page constrain the user to a predefined set of goals (for example, weight loss, muscle gain, endurance) and intensities (low, medium, high). Constraining these fields to a fixed vocabulary keeps the Gemini prompts consistent and makes the stored data easier to review on the administrator dashboard.

10.3 AI Workout Plan Generation Module

`gemini_generator.py` is the core of this module: it builds the prompt described in Chapter 9, invokes the Gemini API, and returns the resulting seven-day plan to `routes.py`, which stores it via `database.py` and renders it through `result.html`.

10.4 Nutrition/Recovery Tip Module

`gemini_flash_generator.py` implements this module, producing a short, goal-relevant nutrition or recovery tip that is displayed alongside the workout plan on `result.html` and stored in the `plans` table's `nutrition_tip` column.

10.5 Feedback and Plan Update Module

`feedback.html` collects the user's free-text feedback; the corresponding route in `routes.py` retrieves the stored original plan and user profile, passes them together with the feedback to `updated_plan.py`, and stores and displays the resulting `updated_plan`.

10.6 Administrator Module

This module spans `admin_login.html`, `admin_auth.py` and `all_users.html`. It authenticates the administrator against environment-configured credentials and, on success, queries every row of the `users` and `plans` tables to render a complete listing for review, together with a logout action that ends the administrator's session.

10.7 Inter-Module Communication

All modules communicate through plain Python function calls orchestrated by `routes.py`, which acts as the single coordination point between the Jinja2 templates, the three Gemini generator modules, and `database.py`. This keeps the dependency graph shallow: no module other than `routes.py` needs to know about more than one of the others.

Chapter 11 – Testing

11.1 Testing Methodology

FitBuddy was tested using a combination of manual functional testing of each route through the browser, targeted unit checks of the database and administrator-authentication modules, and exploratory usability testing with the project team acting as end users. Testing focused on the three critical paths through the system: plan generation, feedback-driven plan update, and administrator access.

11.2 Unit Testing

Module	Test Focus	Result
database.py	Creating and retrieving a user record by user_id	Pass
database.py	Updating an existing plan row with an updated_plan value	Pass
admin_auth.py	Correct credentials return true; incorrect credentials return false	Pass
gemini_generator.py	Prompt string contains all supplied profile fields	Pass

Table 11.1: Representative unit test cases

11.3 Functional / Route Testing

Test Case	Steps	Expected Result	Status
Generate plan with valid input	Submit home-page form with valid values for all fields	Seven-day plan and nutrition tip are displayed and stored	Pass
Generate plan with missing field	Submit form with age left blank	Validation error returned; no Gemini call made	Pass
Submit feedback and view update	Submit feedback text on the feedback page	Updated plan is generated, displayed and stored	Pass
Administrator login (valid)	Submit correct admin credentials	Redirected to the dashboard listing all users	Pass
Administrator login (invalid)	Submit incorrect admin credentials	Login page re-rendered with an error message	Pass
Administrator logout	Click logout from the dashboard	Session ended; dashboard no longer accessible	Pass

Table 11.2: Functional route test cases

11.4 Usability Testing

The single-form layout of the home page was tested with team members unfamiliar with the exact field order; all were able to complete the form and receive a plan without guidance, confirming that the form's labels and dropdown options were self-explanatory.

11.5 Security Testing

The .env file containing the Gemini API key and administrator credentials was confirmed to be excluded from version control via .gitignore. The administrator dashboard route was verified to be inaccessible without first passing through the login check in admin_auth.py.

11.6 Bug Tracking and Resolution

Issues identified during testing — such as an early version of the update-plan route not correctly re-fetching the original plan before calling Gemini — were tracked informally through the team's GitHub repository and resolved before the final build.

Chapter 12 — Results and Discussion

12.1 Summary of Achieved Functionality

All functional requirements listed in Chapter 4 were implemented and verified: personal-information collection, AI-generated seven-day workout plans, AI-generated nutrition/recovery tips, feedback-driven plan regeneration, persistent storage of users and plans, and a secure administrator dashboard with login and logout.

12.2 Observed Plan Quality

Plans generated by Gemini were consistently structured as a clear Day 1 to Day 7 breakdown and varied sensibly with the selected goal and intensity — for example, plans generated with a muscle-gain goal emphasised resistance exercises, while endurance-goal plans emphasised cardiovascular sessions. Updated plans generated from feedback (for example, a request for fewer high-impact exercises) reliably reflected the requested change while retaining the overall structure of the original plan.

12.3 Performance Observations

Operation	Typical Response Time
Home page load	Under 100 ms
Workout plan generation (Gemini call)	2–5 seconds
Nutrition tip generation (Gemini call)	1–2 seconds
Updated plan generation (Gemini call)	2–5 seconds
Administrator dashboard load	Under 200 ms for a small number of users

Table 12.1 Observed response times during local testing

12.4 User Feedback Summary

Informal feedback from team members and reviewers was positive regarding the simplicity of the form and the relevance of the generated plans, with the most common suggestion being to add more predefined goal and intensity options — noted in Chapter 14 as a possible future enhancement.

12.5 Screenshots

Screenshots of the home page, the generated result page, the feedback page and the administrator dashboard are included in the project submission alongside this report and in the project repository's static/images directory.

Chapter 13 — Advantages and Limitations

13.1 Advantages

- Genuine personalisation of the workout plan without the cost of a human trainer.
- A simple, single-form user experience that requires no account registration.
- A built-in feedback loop that lets a user refine their plan in one additional step.
- A lightweight technology stack (FastAPI, SQLite, Jinja2) that is easy to install, run and grade.
- Clear separation of concerns across small, focused Python modules, making the codebase easy to extend.
- Sensitive configuration (API key, administrator credentials) kept out of source control via environment variables.

13.2 Limitations

- Users are identified by a self-entered user ID rather than a password-protected account, so there is no guarantee of exclusive access to a given user's data.
- The quality of a generated plan depends on the Gemini model's output and cannot be manually verified by a certified trainer.
- The plans table stores only one original and one updated plan per user, so a history of multiple feedback iterations is not retained.
- The application depends on an active internet connection and the availability of the Gemini API; it cannot generate plans while offline.
- No mechanism currently tracks whether a user actually follows the generated plan over time.

Chapter 14 — Future Enhancements

14.1 User Accounts and Authentication

Introducing password-protected user accounts (rather than a free-text user ID) would let each user securely access only their own stored plans across multiple sessions and devices.

14.2 Progress Tracking

A dedicated module for logging completed workouts and body-metric changes over time would let FitBuddy show progress trends and feed that history back into future plan generations.

14.3 Conversational Follow-Up Assistant

Extending the Gemini integration with a chat-style interface would allow users to ask follow-up questions about their plan (for example, substituting an exercise) without submitting a full feedback form.

14.4 Additional Goal and Intensity Options

Expanding the predefined lists of fitness goals and workout intensities, based on user feedback gathered in Chapter 12, would widen the range of plans FitBuddy can meaningfully personalise.

14.5 PDF Export

Allowing a user to download their generated plan as a PDF would make it easier to use the plan away from a browser, for example printed and taken to a gym.

14.6 Mobile Application

A companion mobile application, consuming the same FastAPI backend through a small JSON API layer, would let users generate and view plans without opening a desktop browser.

14.7 Multiple Plan History

Extending the plans table (or introducing a plan-history table) to store every regenerated plan, rather than overwriting the previous one, would let users compare successive versions of their plan.

Chapter 15 — Conclusion

FitBuddy demonstrates that a small, well-structured web application built on FastAPI, SQLAlchemy and SQLite can deliver genuinely personalised fitness guidance by delegating the reasoning-heavy task of plan creation to Google's Gemini generative AI models. By keeping the user-facing form to a single page and building a direct feedback-to-regeneration pipeline, the project shows that meaningful personalisation does not require a large team of human coaches or a complex machine-learning pipeline of its own. The administrator dashboard further demonstrates that the same lightweight stack can support basic administrative oversight without additional infrastructure.

The project met all of the objectives set out in Chapter 1: it collects user information, generates and stores AI-driven workout plans and nutrition tips, supports feedback-based refinement, and provides secure administrator access to the underlying data. The limitations identified in Chapter 13, together with the enhancements proposed in Chapter 14, outline a clear path for extending FitBuddy from an academic prototype into a more complete fitness-companion product.

References

1. FastAPI Documentation. Available at: <https://fastapi.tiangolo.com/>
2. SQLAlchemy Documentation. Available at: <https://docs.sqlalchemy.org/>
3. Google AI for Developers — Gemini API Documentation. Available at: <https://ai.google.dev/gemini-api/docs>
4. google-genai Python SDK. Available at: <https://pypi.org/project/google-genai/>
5. Jinja2 Documentation. Available at: <https://jinja.palletsprojects.com/>
6. Uvicorn Documentation. Available at: <https://www.uvicorn.org/>
7. SQLite Documentation. Available at: <https://www.sqlite.org/docs.html>
8. python-dotenv Documentation. Available at: <https://pypi.org/project/python-dotenv/>

Appendix

A.1 Project Repository Structure

```
Fit Buddy/  
  app/  
    __init__.py  
    main.py  
    routes.py  
    database.py  
    admin_auth.py  
    gemini_generator.py  
    gemini_flash_generator.py  
    updated_plan.py  
    schemas.py  
  templates/  
    index.html  
    result.html  
    feedback.html  
    admin_login.html  
    all_users.html  
  static/  
    images/  
      gym_bg_1.jpg  
  .env  
  .env.example  
  .gitignore  
  LICENSE  
  README.md  
  requirements.txt  
  fitbuddy.db
```

A.2 How to Run the Application

```
# 1 Create and activate a virtual environment  
python -m venv venv  
venv\Scripts\activate      # Windows  
source venv/bin/activate   # macOS / Linux  
  
# 2. Install dependencies  
pip install -r requirements.txt  
  
# 3. Configure environment variables in a .env file  
#   (see .env.example — GEMINI_API_KEY, ADMIN_USERNAME, ADMIN_PASSWORD)  
  
# 4. Run the application  
uvicorn app.main:app --reload  
  
# 5. Open in a browser  
http://127.0.0.1:8000
```

A.3 Requirements.txt

```
fastapi  
uvicorn  
jinja2
```

```
sqlalchemy  
python-dotenv  
python-multipart  
google-generativeai
```